

Assignment 1: Informed Search for Maze Solving

Submission date: 5th October, 2026

Greedy Best-First Search (GBFS) and A* Search

Extend the provided maze program (already supporting DFS/BFS) by implementing GBFS and A*. The purpose is to understand how heuristic information guides search.

1. Problem Setup

Maze symbols: A = start, B = goal, # = wall, blank = open cell. Moves: up, down, left, right. Each move costs 1.

Starter code already includes:

- Node class
- Maze loading/representation
- Neighbor generation
- DFS/BFS frontiers
- Path reconstruction and solution display

2. Heuristic

Use Manhattan distance:

$$h(n) = |\text{row} - \text{goal_row}| + |\text{column} - \text{goal_column}|$$

Implement this in the Maze class. It is suitable because movement is horizontal/vertical and every move costs 1.

3. PriorityFrontier

Create PriorityFrontier using Python's heapq. The node with the smallest priority is removed first.

- add(node, priority)
- remove()
- empty()
- contains_state(state)

4. Greedy Best-First Search

Implement solve_greedy() using the priority frontier.

Priority: $f(n) = h(n)$

Expand the node with the smallest heuristic value. Do not add states already explored or already in the frontier. Reconstruct the path when B is reached.

Record: explored states, path cost/moves, maximum frontier size, and runtime. GBFS does not guarantee the shortest path.

5. A* Search

Implement `solve_astar()` using the priority frontier.

Priority: $f(n) = g(n) + h(n)$

$g(n)$ = actual cost from A to n; $h(n)$ = Manhattan estimate from n to B. Start with $g(A)=0$. Maintain the best known cost for each state and update it when a cheaper route is found.

Record: explored states, path cost, maximum frontier size, and runtime. With unit-cost moves and Manhattan distance, A* should find a shortest path.

6. Experimental Comparison

Run BFS, GBFS, and A* on the same three mazes:

- Simple maze
- Maze with multiple possible paths
- Maze with dead ends or misleading routes

Maze	Algorithm	Found?	Explored	Path Cost / Moves	Max Frontier	Runtime
maze1.txt	BFS					
maze1.txt	GBFS					
maze1.txt	A*					
maze2.txt	BFS					
maze2.txt	GBFS					
maze2.txt	A*					
maze3.txt	BFS					
maze3.txt	GBFS					
maze3.txt	A*					

7. Short Questions

1. What is the main difference between BFS, GBFS, and A*?
2. What do $g(n)$, $h(n)$, and $f(n)$ represent?
3. Why does GBFS use $f(n)=h(n)$?
4. Why can GBFS return a non-shortest path?
5. Why does A* use both $g(n)$ and $h(n)$?
6. Why is Manhattan distance suitable for this maze?
7. What is an admissible heuristic?
8. Why can A* find an optimal path with a suitable heuristic?

8. Submission

- Complete `maze.py`
- Three maze files

- Solution screenshots/images
- Short report with results and answers
- README.txt with run instructions

9. Rules

- No external pathfinding libraries.
- Standard libraries such as sys, heapq, and time are allowed.
- Do not hard-code the route.
- Program must work with different valid mazes.
- Comment your code and handle mazes with no solution.

10. Optional Commands

```
python maze.py maze1.txt bfs  
python maze.py maze1.txt greedy  
python maze.py maze1.txt astar
```

Marks: 20

Component	Marks
Priority Frontier	3
Manhattan Heuristic	2
GBFS	4
A*	5
Path Reconstruction / Output	2
Experimental Comparison	2
Questions / Discussion	2